



Online Food Ordering System

SQL Database Management System Case Study

PRESENTED BY SHRAVAN JADHAV



Introduction

The **Online Food Ordering System** is a relational database project built using SQL to manage the full lifecycle of food ordering – from customer profiles and restaurant menus to order tracking and reviews.

Database Design

Structured schema with normalized tables and enforced referential integrity

SQL Query Writing

Practical queries for filtering, joining, and aggregating real-world data

Business Analysis

Actionable insights on revenue, customer behavior, and restaurant performance

Project Objectives



Relational Database Design

Design a normalized schema with proper entity relationships and foreign keys



Customer Management

Store and manage customer profiles, contact details, and ordering history



Restaurant & Menu Data

Maintain restaurant details, cuisine types, menus, and item pricing



Order Tracking

Record orders, quantities, prices, and delivery status accurately



Customer Reviews

Capture and analyze customer feedback and restaurant ratings

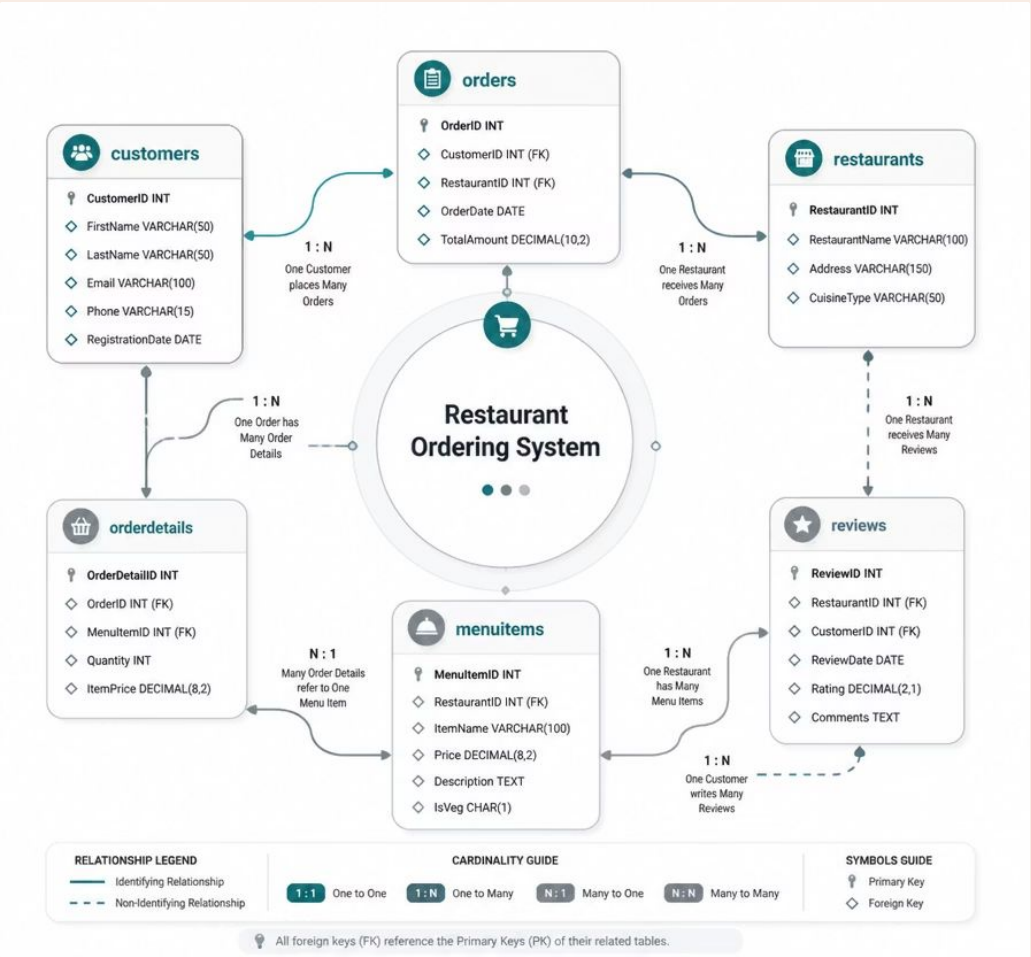


Business Insights

Generate reports on revenue, popular items, and customer spending

Database Schema

Six normalized tables linked through foreign keys, forming a complete relational model for food ordering.



Entity Relationships

- **Customers** → **Orders** – One customer places many orders
- **Restaurants** → **MenuItems** – One restaurant offers many menu items
- **Orders** → **OrderDetails** – One order contains many line items
- **MenuItems** → **OrderDetails** – Items referenced across orders
- **OrderDetails** → **Reviews** – Each order item can be reviewed

Database Tables

Customers	Stores customer ID, name, email, phone, and address
Restaurants	Holds restaurant ID, name, cuisine type, and location
MenuItems	Lists menu items with name, category, price, and availability
Orders	Records order ID, customer ID, restaurant ID, and order date
OrderDetails	Links orders to menu items with quantities and item prices
Reviews	Captures ratings and written feedback linked to order details

SQL Concepts Used



DDL

CREATE TABLE, ALTER TABLE



DML

INSERT, UPDATE, DELETE



Filtering

SELECT, WHERE, LIKE



Logic

CASE expressions



Aggregation

GROUP BY, HAVING, COUNT, SUM, AVG



Sorting

ORDER BY, LIMIT



Joins

INNER JOIN, LEFT JOIN



Subqueries

Nested SELECT statements

SQL Query #1: Select Menu Items Under \$20

This query finds all food items that cost less than \$20

SELECT *

Get all columns from the table

FROM MenuItems

Look in the MenuItems table

WHERE Price < 20

Only show items where the price is less than \$20

SQL Code

```
SELECT *  
FROM MenuItems  
WHERE Price < 20;
```

SQL Query #2: Orders Between \$30-\$50 in 2024

This query finds all orders that cost between \$30 and \$50 and were placed in 2024

SELECT *

Get all order information

FROM Orders

Look in the Orders table

WHERE TotalAmount BETWEEN 30 AND 50

Filter orders with amounts between \$30 and \$50

AND YEAR(OrderDate)=2024

Also filter to only orders from the year 2024

SQL Code

```
SELECT *  
FROM Orders  
WHERE TotalAmount BETWEEN 30 AND 50  
AND YEAR(OrderDate)=2024;
```

SQL Query #3: Find Italian Restaurants

This query finds all restaurants that serve Italian cuisine

SELECT *

Get all restaurant information

FROM Restaurants

Look in the Restaurants table

WHERE CuisineType LIKE '%Italian%'

Filter to only restaurants where the cuisine type contains the word 'Italian' (the % symbols mean 'any characters before or after')

SQL Code

```
SELECT *  
FROM Restaurants  
WHERE CuisineType LIKE '%Italian%';
```

SQL Query #4: Calculate Discounted Prices

This query applies a 10% discount to items priced over \$15

Step by step

- `SELECT ItemName, Price` – Get the item name and original price
- `CASE WHEN Price > 15 THEN ROUND(Price*0.90,2) ELSE Price END AS DiscountedPrice` – Use a CASE statement to check: if price is greater than \$15, multiply by 0.90 (which gives 10% off), otherwise keep the original price. ROUND rounds to 2 decimal places
- `FROM MenuItems` – Look in the MenuItems table

SQL Code

```
SELECT
  ItemName,
  Price,
  CASE
    WHEN Price > 15
    THEN ROUND(Price * 0.90, 2)
    ELSE Price
  END AS DiscountedPrice
FROM MenuItems;
```

SQL Query #5: Find High-Spending Customers

This query finds customers who have spent more than \$1000 in total orders

SELECT * FROM Customers

Get all customer information

WHERE CustomerID IN (...)

Filter to only customers whose ID appears in the subquery results

Subquery

The inner query groups all orders by customer and sums their total amounts, then only keeps customers with a sum greater than \$1000

SQL Code

```
SELECT *  
FROM Customers  
WHERE CustomerID IN  
(  
  SELECT CustomerID  
  FROM Orders  
  GROUP BY CustomerID  
  HAVING SUM(TotalAmount) > 1000  
);
```

SQL Query #6: Total Orders Per Restaurant

This query counts how many orders each restaurant has received

How it works

- **SELECT r.RestaurantID, r.RestaurantName, COUNT(o.OrderID) AS TotalOrders** – Get restaurant ID, name, and count the number of orders
- **FROM Restaurants r LEFT JOIN Orders o ON r.RestaurantID=o.RestaurantID** – Start with all restaurants and connect them to their orders (LEFT JOIN means we keep all restaurants even if they have no orders)
- **GROUP BY r.RestaurantID, r.RestaurantName** – Group the results by restaurant so we get one row per restaurant with the count of their orders

SQL Query

```
SELECT
r.RestaurantID,
r.RestaurantName,
COUNT(o.OrderID) AS TotalOrders
FROM Restaurants r
LEFT JOIN Orders o
ON r.RestaurantID=o.RestaurantID
GROUP BY
r.RestaurantID,
r.RestaurantName;
```

SQL Query #7: Restaurant Revenue Over \$500

This query finds restaurants that have earned more than \$500 in total orders

SELECT r.RestaurantID, r.RestaurantName, SUM(o.TotalAmount) AS TotalRevenue

Get restaurant ID, name, and sum all their order amounts

FROM Restaurants r JOIN Orders o ON r.RestaurantID=o.RestaurantID

Connect restaurants to their orders (INNER JOIN means we only get restaurants that have orders)

GROUP BY r.RestaurantID, r.RestaurantName

Group by restaurant to get one row per restaurant with their total revenue

HAVING SUM(o.TotalAmount)>500

Filter to only show restaurants where the total revenue is greater than \$500

SQL Code

```
SELECT
  r.RestaurantID,
  r.RestaurantName,
  SUM(o.TotalAmount) AS TotalRevenue
FROM Restaurants r
JOIN Orders o
  ON r.RestaurantID=o.RestaurantID
GROUP BY
  r.RestaurantID,
  r.RestaurantName
HAVING SUM(o.TotalAmount)>500;
```

SQL Query #8: Top 5 Highest-Rated Restaurants

This query finds the 5 restaurants with the highest average ratings

How it works

- `SELECT r.RestaurantID, r.RestaurantName, ROUND(AVG(rv.Rating),2) AS AvgRating` – Get restaurant ID, name, and calculate the average rating (rounded to 2 decimal places)
- `FROM Restaurants r JOIN Reviews rv ON r.RestaurantID=rv.RestaurantID` – Connect restaurants to their reviews
- `GROUP BY r.RestaurantID, r.RestaurantName` – Group by restaurant to get one average rating per restaurant
- `ORDER BY AvgRating DESC` – Sort by average rating in descending order (highest first)
- `LIMIT 5` – Only show the top 5 restaurants

SQL Query

```
SELECT
  r.RestaurantID,
  r.RestaurantName,
  ROUND(AVG(rv.Rating),2) AS AvgRating
FROM Restaurants r
JOIN Reviews rv
  ON r.RestaurantID=rv.RestaurantID
GROUP BY
  r.RestaurantID,
  r.RestaurantName
ORDER BY AvgRating DESC
LIMIT 5;
```

SQL Query #9: Orders with Customer Names

This query gets all orders with customer names, dates, and amounts

- `SELECT o.OrderID, CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.OrderDate, o.TotalAmount` – Get order ID, combine first and last name into one customer name field, order date, and total amount
- `FROM Orders o JOIN Customers c ON o.CustomerID=c.CustomerID` – Connect orders to customers using their customer ID
- The `CONCAT` function combines the first name and last name with a space between them

SQL Code

```
SELECT
  o.OrderID,
  CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
  o.OrderDate,
  o.TotalAmount
FROM Orders o
JOIN Customers c
  ON o.CustomerID=c.CustomerID;
```

SQL Query #10: Menu Items and Order Details

This query gets all menu items and their associated order details, including items that have never been ordered.

How it works

- `SELECT m.MenuItemID, m.ItemName, od.OrderID, od.Quantity, od.ItemPrice` – Get menu item ID, item name, order ID, quantity ordered, and the price paid
- `FROM MenuItems m LEFT JOIN OrderDetails od ON m.MenuItemID=od.MenuItemID` – Start with all menu items and connect them to order details (LEFT JOIN means we keep all items even if they've never been ordered)
- If an item has never been ordered, the order-related columns will show `NULL` (empty)

SQL Code

```
SELECT
  m.MenuItemID,
  m.ItemName,
  od.OrderID,
  od.Quantity,
  od.ItemPrice
FROM MenuItems m
LEFT JOIN OrderDetails od
  ON m.MenuItemID = od.MenuItemID;
```

SQL Query #11: Menu Items Count Per Restaurant

This query gets the total number of menu items available in each restaurant

**SELECT r.RestaurantID,
r.RestaurantName,
COUNT(m.MenuItemId) AS
TotalMenuItems**

Get restaurant ID, name, and count how many menu items they have

**FROM Restaurants r LEFT
JOIN MenuItems m ON
r.RestaurantID=m.RestaurantID**

Start with all restaurants and connect them to their menu items (LEFT JOIN means we keep all restaurants even if they have no items)

**GROUP BY r.RestaurantID,
r.RestaurantName**

Group by restaurant to get one row per restaurant with their item count

SQL Code

```
SELECT
  r.RestaurantID,
  r.RestaurantName,
  COUNT(m.MenuItemId) AS TotalMenuItems
FROM Restaurants r
LEFT JOIN MenuItems m
  ON r.RestaurantID=m.RestaurantID
GROUP BY
  r.RestaurantID,
  r.RestaurantName;
```

SQL Query #12: Highly-Rated Restaurants

This query finds restaurants with ratings above 4.0 from at least 3 different customers

The subquery (inner SELECT)

```
SELECT RestaurantID FROM  
Reviews WHERE Rating>4 GROUP  
BY RestaurantID HAVING  
COUNT(DISTINCT CustomerID)>=3
```

This finds restaurants where at least 3 different customers gave a rating above 4.0.

The main query

```
SELECT r.RestaurantID,  
r.RestaurantName FROM  
Restaurants r JOIN Reviews rv ON  
r.RestaurantID=rv.RestaurantID  
WHERE r.RestaurantID IN (...)
```

Get restaurant details for those restaurants that passed the subquery filter.

GROUP BY r.RestaurantID, r.RestaurantName

Group to get one row per restaurant.

SQL Code

```
SELECT  
r.RestaurantID,  
r.RestaurantName  
FROM Restaurants r  
JOIN Reviews rv  
ON r.RestaurantID=rv.RestaurantID  
WHERE r.RestaurantID IN  
(  
SELECT RestaurantID  
FROM Reviews  
WHERE Rating>4  
GROUP BY RestaurantID  
HAVING COUNT(DISTINCT CustomerID)>=3  
)  
GROUP BY  
r.RestaurantID,  
r.RestaurantName;
```

SQL Query #13: Complete Order Details

This query retrieves a comprehensive view of all orders, combining information from customers, restaurants, menu items, and order specifics to provide a full picture of each transaction.

Joining Orders and Customers

```
FROM Orders o JOIN Customers c ON o.CustomerID = c.CustomerID
```

Connects orders to customer details to retrieve customer names and contact information.

Joining with Restaurants

```
JOIN Restaurants r ON o.RestaurantID = r.RestaurantID
```

Links each order to the restaurant that prepared it, providing restaurant names and cuisine types.

Retrieving Order Items

```
JOIN OrderDetails od ON o.OrderID = od.OrderID JOIN MenuItem mi ON od.MenuItemID = mi.MenuItemID
```

Combines order details with menu item information to show which specific items were ordered, their quantities, and names.

Calculating Total Item Price

```
SELECT ..., (od.Quantity * od.ItemPrice) AS TotalItemPrice
```

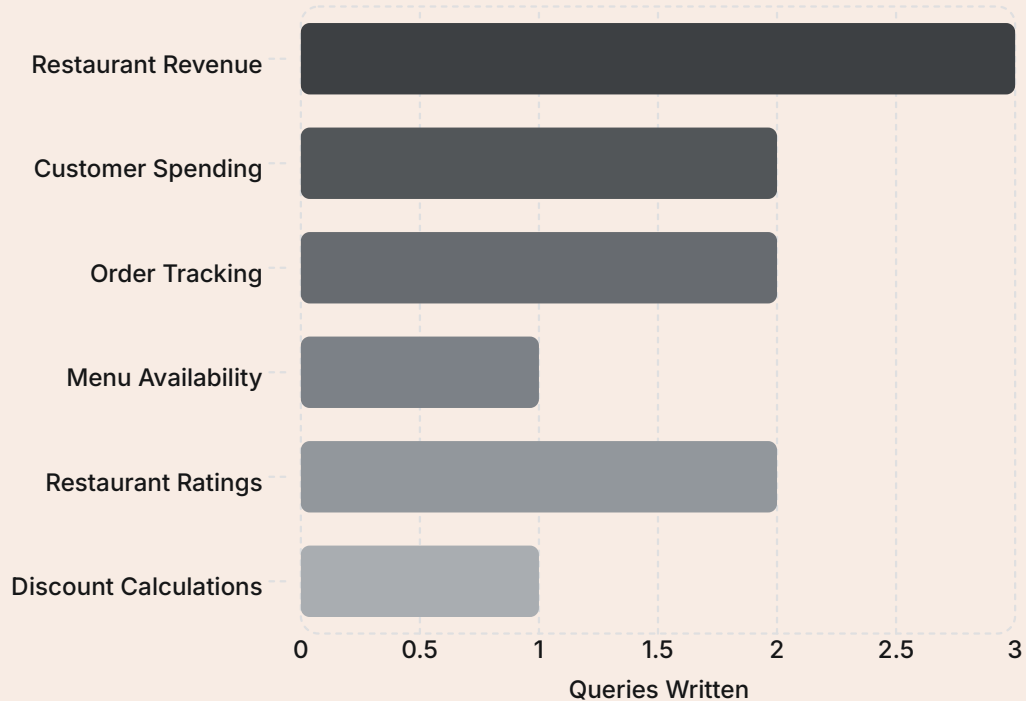
Calculates the total price for each individual menu item within an order.

SQL Code

```
SELECT
  o.OrderID,
  c.CustomerName,
  r.RestaurantName,
  o.OrderDate,
  mi.MenuItemName,
  od.Quantity,
  od.ItemPrice,
  (od.Quantity * od.ItemPrice) AS TotalItemPrice
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Restaurants r ON o.RestaurantID = r.RestaurantID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN MenuItem mi ON od.MenuItemID = mi.MenuItemID;
```

SQL Analysis Performed

Analysis Type



Analyses Completed

- Top-rated restaurants by average review score
- Restaurant-wise total revenue
- Customers spending over ₹1,000
- Order count per restaurant
- Menu item availability status
- Discount calculations on order totals
- Customer ordering history
- Restaurant performance rankings